CAR RENTAL

Reservation & Vehicle Return Management System

FINAL PRESENTATION – REVIEW III

Working Python + Tkinter + MySQL Application

Live CRUD • Availability • Reports • Validation

PRESENTED BY

Koneri Mithul
25WU0102129
AIML Panthers
Guide: Dr. Kiran Mayee Adavala

Project Objective & Final Deliverable

THE PROBLEM

Manual/disconnected rental tracking can cause double-boo

THE SOLUTION

A normalized multi-branch rental database with a live Pyth

REVIEW III GOAL

Demonstrate working insertion, deletion and viewing of records

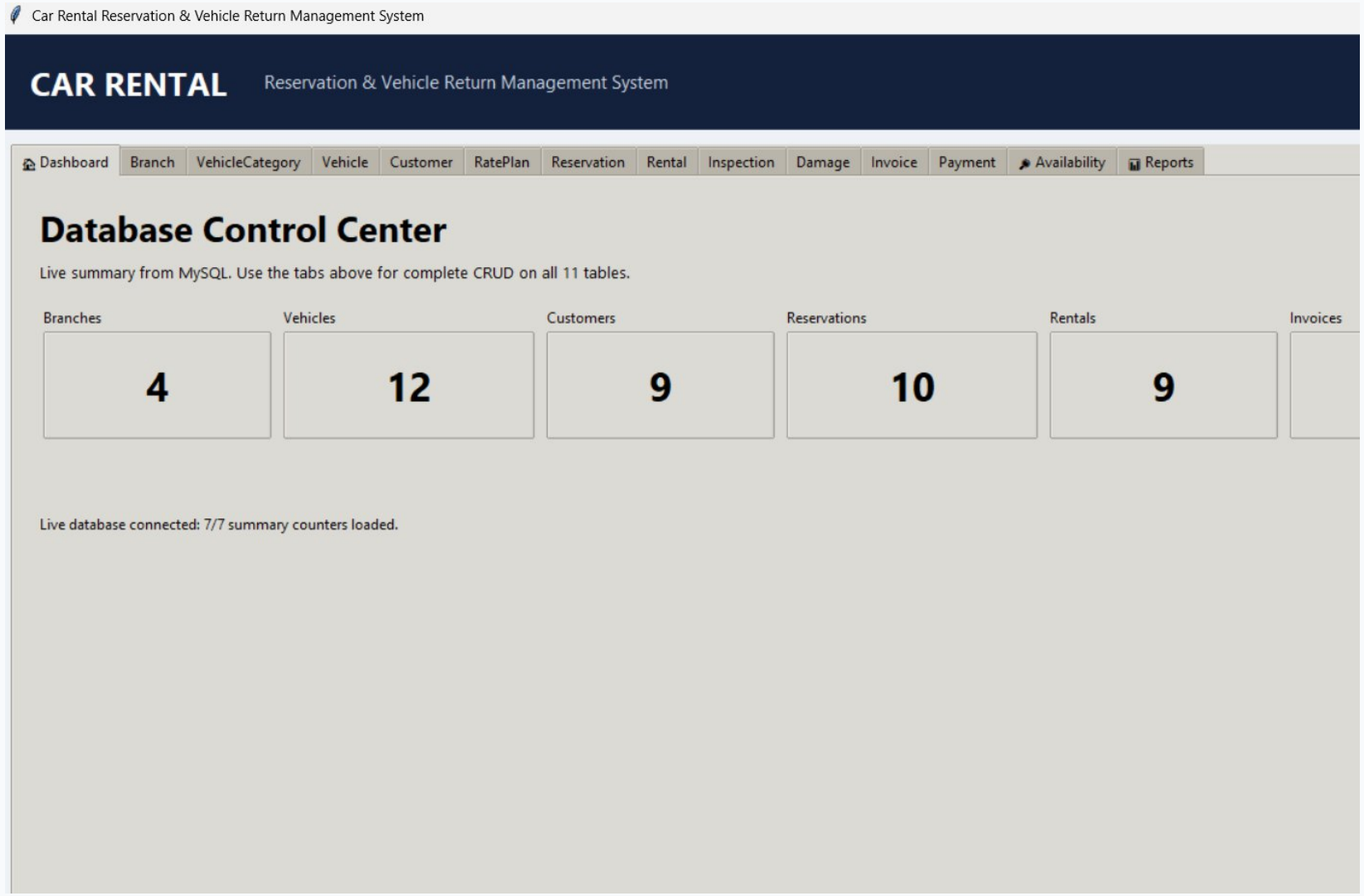
Final system combines database design + implementation + live application.

UI includes all 11 project tables plus availability and reporting.

CORE FLOW

Customer → Reservation → Rental → Inspection → Damage → Invoice → Payment

Working UI – Dashboard



LIVE DATABASE

7/7 summary counters loaded

MODULES

11 table tabs
Availability
Reports
Live MySQL connection

CRUD READY

INSERT / UPDATE / DELETE / VIEW
with refresh after each operation

Live CRUD Demonstration Plan

INSERT

Create a test Customer

VIEW

Refresh and show it

VERIFY

SELECT in MySQL

DELETE

Delete the test record

VERIFY

SELECT again – no row

MYSQL VERIFICATION

```
USE car_rental_db;

SELECT * FROM customer;

SELECT * FROM customer
WHERE license_no='TESTDL1001';
```

Expected: row exists after INSERT and no row after DELETE.

Validation & Database Integrity

Referential integrity

PK/FK relationships prevent invalid references.

Licence validation

Pickup is blocked for an invalid/expired licence.

No overlap

Rental issue logic checks conflicting rental periods.

Return validation

Return time and odometer rules protect data quality.

Safe charges

Charges/payments use non-negative or positive checks.

Transactions

Commit/rollback keeps multi-step operations consistent.

Availability & Reporting

AVAILABILITY SEARCH

Search by pickup/return dates, branch and vehicle category.

Output: vehicles available for the requested rental window.

REPORTS

Fleet availability
Overdue rentals
Customer rental history
Branch revenue
Vehicle utilization
Damage report

Final 10-Minute Demonstration Sequence

1. Dashboard – prove live MySQL connection
2. Customer – INSERT a test record
3. Customer – VIEW the inserted record
4. MySQL Workbench – SELECT to prove the database change
5. Customer – DELETE the test record
6. MySQL Workbench – SELECT again to prove deletion
7. Show Vehicle / Reservation / Rental CRUD screens
8. Demonstrate Availability search
9. Demonstrate Reports
10. Conclude with database + UI + integrity

CLOSING LINE

"Our Python UI is directly connected to MySQL, and changes are reflected in

THANK YOU

FINAL TAKEAWAY

A normalized MySQL database + working Python application supporting rental operations, CRUD, integrity validation, availability and reporting.

Koneri Mithul • 25WU0102129 • AIML Panthers